

Relatório de Análise Estática de Código

Projeto Observação

Foram utilizadas as ferramentas SonarQube, Checkstyle, PMD e SpotBugs. O SonarQube foi utilizado para análise de qualidade, segurança e manutenibilidade. O Checkstyle para padronização de código. O PMD para identificação de más práticas. O SpotBugs para detecção de possíveis falhas de implementação.

1. Panorama Geral — Resultados por Ferramenta

Ferramenta	Módulo	Issues	Status	Principais Achados
SonarQube	Backend Java	29	✓ Passou	Fuso horário, exceções genéricas, literais duplicados
SonarQube	Frontend Next.js	107	✓ Passou*	globalThis, props readonly, ternários aninhados
Checkstyle 9.3	Backend Java	557 erros	⚠ Atenção	Javadoc ausente, FinalParameters, HiddenField
PMD 7.17.0	Backend Java	0	✓ Aprovado	Nenhum problema identificado
SpotBugs	Backend Java	Alertas	i Informativo	Campos mutáveis, construtores com lógica

PMD Results

The following document contains the results of PMD (<https://pmd.github.io>) 7.17.0.

PMD found no problems in your source code.

© 2026



(<https://maven.apache.org/>)

2. Métricas SonarQube — Módulo Backend (AEP)

Métrica	Valor	Avaliação do SonarQube / Risco
Portão de Qualidade	Passou	✓ Aprovado geral
Linhas de Código	1.493 LOC	Projeto de porte pequeno-médio
Segurança / Confiabilidade	9 issues (A)	⚠ Alerta: Uso de LocalDateTime.now() sem ZoneId
Manutenibilidade / Débito	20 issues / 3h47m	✓ Bom: Exceções genéricas e literais duplicados
Cobertura de Testes	0,0%	🔴 CRÍTICO — Risco extremo de regressões
Duplicações / Complexidade	0,0% / 137	✓ Ótimo: Sem código clonado, complexidade baixa

3. Análise Crítica de Código — Backend Java (SonarQube + SpotBugs)

??

A base de dados incorporada deve ser usada apenas para fins de avaliação. Ele não suporta escalonamento, atualização para uma nova versão do SonarQube Server ou migração para outro mecanismo de banco de dados.

??

SonarQube

community

Projetos

Questões

Regras

Perfis qualidade

Portões qualidade

Administração

Mais, mais

??

??

??

??

A.A

A.A

AEP

Projeto

??

Para se beneficiar de mais recursos do SonarQube Community Build, [monte análises no seu CI favorito](#). O.

??

AEP

/

Visão GERAL

Visão GERAL

1,5k Linhas de Código · Versão 0.0.1-SNAPSHOT · Última análise há 2 minutos

Definir como página inicial

??

Portão Qualidade

??

Passou

Novo Código

Código Geral

Segurança

0 Questões abertas

A.A

Confiabilidade

9 Questões abertas

A.A

Manutenibilidade

20 Questões abertas

A.A

Questões aceitas

0

Questões válidas que não foram corrigidas

Cobertura

0,0%

Em diante 419 linhas a percorrer.

Duplicações

0,0%

Em diante 1,8k linhas.

Ponto de acesso de segurança

11

E.E

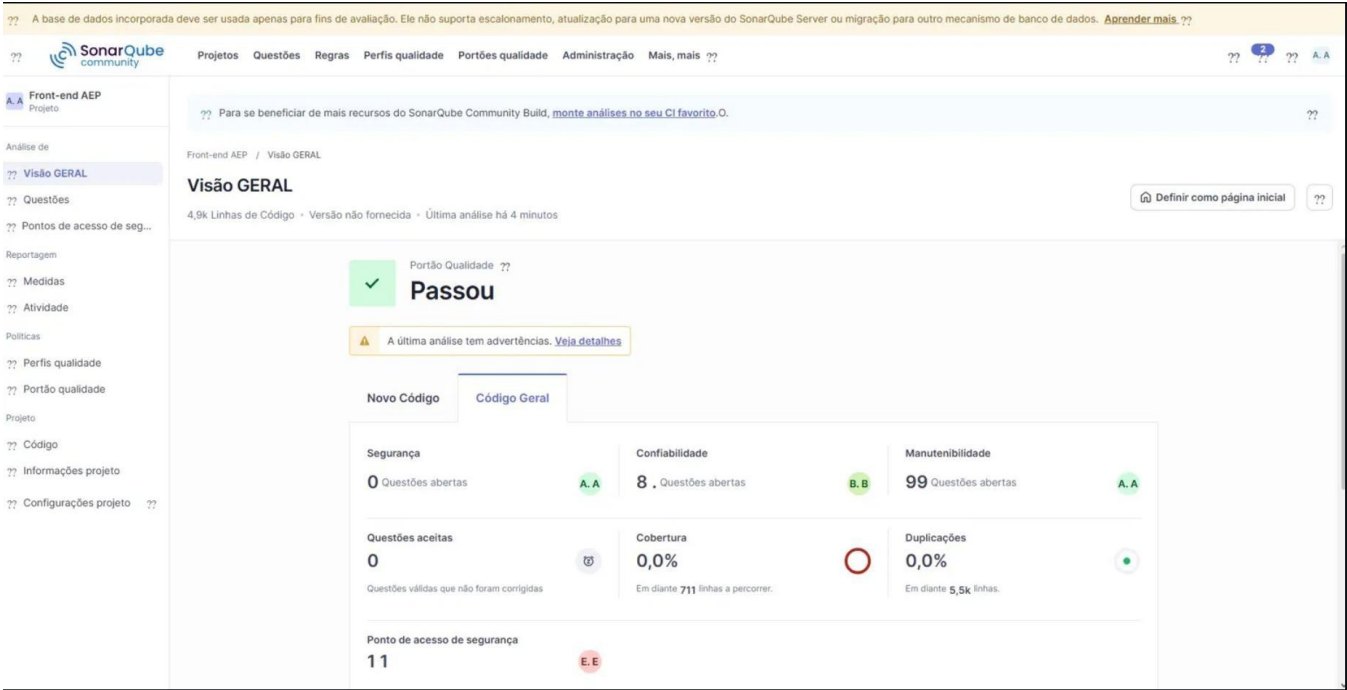
2.1 Visão Geral do Projeto AEP

Problema & Impacto no Sistema	Causa (Por que apareceu?)	Correção Recomendada & Esforço
LocalDateTime.now() sem ZoneId Impacto: Divergência de horários e logs entre ambientes (Docker UTC vs Dev Local).	Chamadas dependentes do fuso horário default da JVM em runtime.	Substituir todas as ocorrências por <code>LocalDateTime.now(ZoneId.of("UTC"))</code> . Esforço: 1-2h.
Exceções Genéricas (RuntimeException) Impacto: Controllers não distinguem erros HTTP de forma limpa (ex: 404 de 403).	Uso de exceção genérica como atalho rápido nos Services e SecurityConfig.	Criar hierarquia (ex: <code>EntityNotFoundException</code> , <code>AccessDeniedException</code>). Esforço: 1-2 dias.
Campos não-final nos Services Impacto: Risco de mutabilidade e 56 violações "HiddenField" no Checkstyle.	Omissão do qualificador final em variáveis injetadas por construtor.	Adicionar <code>private final</code> nos campos em <code>AuthService</code> e <code>SecurityConfig</code> . Esforço: minutos.
Literais de String Duplicados Impacto: Risco de erros e inconsistências de segurança ao alterar rotas ou roles.	Strings como "SERVIDOR" repetidas várias vezes diretamente nas classes.	Extrair valores repetidos para constantes estáticas da classe. Esforço: 30 minutos.

4. Análise Crítica de Código — Frontend Next.js (SonarQube)

Problema & Impacto no Sistema	Causa (Por que apareceu?)	Correção Recomendada & Esforço
window vs globalThis (8 ocorrências) Impacto: Risco de ReferenceError no Server-Side Rendering (SSR) do Next.js.	Acesso direto ao objeto window, indisponível durante renderização no servidor.	Substituir chamadas diretas a window por <code>globalThis.window</code> . Esforço: minutos.
Props React sem Readonly Impacto: Permite que componentes filhos modifiquem props dos pais de forma indevida.	Tipagem TypeScript incompleta sem restrição de escrita nas interfaces.	Utilizar <code>Readonly<Props></code> ou qualificador <code>readonly</code> nas interfaces. Esforço: 2h.
Promise.reject() vs throw Impacto: Reduz a legibilidade do código assíncrono e tratamento de exceções.	Uso de Promise explícito no interceptor Axios em vez de tratamento nativo.	Substituir <code>return Promise.reject(error)</code> por <code>throw error</code> . Esforço: minutos.

Problema & Impacto no Sistema	Causa (Por que apareceu?)	Correção Recomendada & Esforço
Ternários Aninhados no JSX Impacto: Leitura complexa do HTML gerado e maior carga cognitiva na manutenção.	Implementação de regras de renderização condicional direto na árvore JSX.	Extrair lógicas complexas para funções auxiliares ou subcomponentes. Esforço: 2-3h.



5. Análise Estética e Padronização — Checkstyle 9.3

Regra Violada	Qtd.	Por que apareceu?	Correção Recomendada
MissingJavadoc	170	Falta de documentação padrão nos métodos e variáveis da aplicação.	Adicionar Javadoc progressivamente em interfaces e serviços críticos.
FinalParameters	140	Regra sun_checks.xml obriga parâmetros imutáveis nos métodos.	Adicionar final aos parâmetros ou migrar para google_checks.xml.
DesignForExtension	72	Incompatibilidade da regra de herança com proxies CGLIB do Spring.	Suprimir a regra para classes Spring ou documentar métodos públicos.
HiddenField	56	Parâmetros de construtores com mesmo nome das variáveis membro.	Corrigir os campos de injeção dos Services para final (item 3).
LineLength	64	Limite rígido de 80 caracteres herdado das regras padrão antigas.	Configurar o Checkstyle para aceitar 120 ou 150 caracteres.
MagicNumber	7	Números inteiros fixos (prazos, tamanho de senha) soltos no código.	Extrair literais para constantes declarativas claras. Fácil correção.

6. Cobertura de Testes — Risco Crítico (0,0%)

● **ALERTA: Cobertura de testes zerada (0,0%) no backend e frontend. Risco severo de quebras.**

- **Por que apareceu:** Desenvolvimento acelerado priorizando funcionalidades em detrimento dos testes automatizados.
- **Impacto real:** Qualquer refatoração (incluindo as correções sugeridas neste relatório) pode introduzir regressões indetectáveis.
- **Estratégia corretiva:** Começar com testes unitários em classes críticas (SolicitacaoService, AuthService), progredindo para MockMvc nos controllers.

7. Tabela de Prioridades Consolidada

Prio	Problema Crítico	Impacto de Negócio	Esforço	Ferramenta
P0	Cobertura de testes zerada (0,0%)	Regressões não detectadas no build	Alto (Semanas)	SonarQube (Quality Gate)
P1	LocalDateTime.now() sem ZoneId	Divergência de fusos em instâncias	Baixo (Horas)	SonarQube java:S8688

Prio	Problema Crítico	Impacto de Negócio	Esforço	Ferramenta
P1	Exceções RuntimeException genéricas	Código HTTP indefinido na API	Médio (1-2 dias)	SonarQube (Backend)
P1	Campos não-final em Services	Inconsistência e mutabilidade	Baixo (Minutos)	SpotBugs + Checkstyle
P2	Literais duplicados / Date no JWT	Manutenção complexa e lenta	Baixo (Horas)	SonarQube (Backend)
P3	Ajustes Checkstyle & Next.js UI	Tipagem TypeScript e formatação	Médio (Dias)	Checkstyle / Sonar FE

8. Conclusão

O projeto Observação possui uma excelente arquitetura em camadas e passou em ambos os portões de qualidade do SonarQube sem duplicações ou brechas graves de segurança. Entretanto, a total ausência de testes e o uso inadequado de fusos horários e exceções genéricas fragilizam sua sustentabilidade. As ações recomendadas de nível P1 devem ser aplicadas de imediato.

Fim do Relatório Consolidado (3 Páginas) · SonarQube | Checkstyle 9.3 | PMD | SpotBugs